125. Valid Palindrome Easy

Acceptance: 52.5% | LeetCode

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string `s`, return `true` *if it is a****palindrome****, or* `false` *otherwise*.

Example 1:

```
**Input:** s = "A man, a plan, a canal: Panama"
**Output:** true
**Explanation:** "amanaplanacanalpanama" is a
palindrome.
```

Example 2:

```
**Input:** s = "race a car"
**Output:** false
**Explanation:** "raceacar" is not a palindrome.
```

Example 3:

```
**Input:** s = " "
**Output:** true
**Explanation:** s is an empty string "" after
removing non-alphanumeric characters.
Since an empty string reads the same forward and
backward, it is a palindrome.
```

Constraints:

- `1 <= s.length <= 2 * 105`
- `s` consists only of printable ASCII characters.

Claude Opus Solution

Valid Palindrome

1. Intuition

A palindrome reads the same forwards and backwards. The key insight is that we only care about **alphanumeric characters** (ignoring case and punctuation).

The natural way to check a palindrome is to compare characters from **both ends moving inward**. If every matching pair is equal, it's a palindrome. This is the classic **Two Pointers** pattern.

The twist here: we must **skip non-alphanumeric characters** as we go, and **compare case-insensitively**.

> ⚠️ Note: There's a bug in the reference `solution2.py` — when characters match, it does `right += 1` instead of `right -= 1`. The solution below fixes this.

---

2. Approach (Pseudocode)

```
left  = 0
right = len(s) - 1

while left < right:
    skip left  forward  while s[left]  is NOT alphanumeric
    skip right backward while s[right] is NOT alphanumeric

    if both pointers still valid (left < right):
        if lowercase(s[left]) != lowercase(s[right]):
            return False
        move both pointers inward

return True
```

---

3. Python Solution

```python
class Solution:
    def isPalindrome(self, s: str) -> bool:
        # Two pointers starting at the outer edges of the string
        left, right = 0, len(s) - 1

        while left < right:
            # Skip any non-alphanumeric characters from the left side
            # (spaces, punctuation, etc. don't count toward the palindrome)
```

```
            while left < right and not s[left].isalnum():
                left += 1

            # Skip any non-alphanumeric characters from the right side
            while left < right and not s[right].isalnum():
                right -= 1

            # Now both pointers are on alphanumeric characters.
            # Compare them case-insensitively. If they differ, not a palindrome.
            if s[left].lower() != s[right].lower():
                return False

            # Characters matched — move both pointers inward to continue.
            left += 1
            right -= 1

        # All comparisons passed -> it's a valid palindrome
        return True
```

Why the inner `left < right` guard matters

The inner while loops check `left < right` to avoid the pointers crossing past each other when, e.g., the remaining characters are all non-alphanumeric. This prevents index-related logic errors and keeps the comparison safe.

---

4. Complexity Analysis

Metric	Value	Explanation
Time	`O(n)`	Each pointer moves only inward and never backward, so we touch each character at most once.
Space	`O(1)`	We use only two pointer variables; no extra string/array is allocated.

This is **optimal** in both time and space.

---

5. Alternative Approaches

A) Filter + Reverse (solution1 style) — clean but uses extra space

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        # Keep only alphanumeric chars, normalize to lowercase
        cleaned = [c.lower() for c in s if c.isalnum()]
        # Compare the list with its reverse
        return cleaned == cleaned[::-1]
```

- **Time:** `O(n)` **Space:** `O(n)` (builds a new filtered string/list).
- More readable, but allocates extra memory — not truly `O(1)`.

B) Filter + Two Pointers on cleaned string

Pre-filter the string, then run two pointers. Simpler logic but still `O(n)` space.

Key Takeaways

1. **Two Pointers from both ends** is the go-to pattern for palindrome / symmetric-comparison problems.
2. When extra characters must be ignored, **skip them in place** rather than building a new string — this preserves `O(1)` space.
3. Always guard inner skip-loops with `left < right` to avoid pointer crossover bugs.
4. Watch out for the subtle `right -= 1` vs `right += 1` bug — moving the wrong direction causes infinite loops or wrong answers.

Codex Solution

1. Intuition

A palindrome reads the same from both ends.

However, this problem asks us to ignore:

- uppercase vs lowercase differences
- non-alphanumeric characters like spaces, commas, colons, etc.

So instead of building a cleaned version of the string, we can use the **two pointers pattern**:

- One pointer starts at the beginning.
- One pointer starts at the end.
- Move both pointers toward the center.
- Skip any character that is not alphanumeric.
- Compare valid characters in lowercase form.

If all matching characters are equal, the string is a valid palindrome.

---

2. Approach

Pseudocode

```
function isPalindrome(s):
    left = 0
    right = length of s - 1

    while left < right:
        while left < right and s[left] is not alphanumeric:
            left += 1

        while left < right and s[right] is not alphanumeric:
            right -= 1

        if lowercase(s[left]) != lowercase(s[right]):
            return false

        left += 1
        right -= 1

    return true
```

---

3. Python Solution

```python
class Solution:
    def isPalindrome(self, s: str) -> bool:
        # Initialize two pointers:
        # left starts from the beginning,
        # right starts from the end.
        left = 0
        right = len(s) - 1

        # Continue until the two pointers meet or cross.
        while left < right:

            # Move left pointer forward until it points to an alphanumeric character.
            while left < right and not s[left].isalnum():
                left += 1

            # Move right pointer backward until it points to an alphanumeric character.
            while left < right and not s[right].isalnum():
                right -= 1

            # Compare the lowercase versions of both characters.
            # If they are different, this cannot be a palindrome.
            if s[left].lower() != s[right].lower():
                return False

            # Move both pointers inward after a successful comparison.
            left += 1
            right -= 1
```

```
        # If no mismatch was found, the string is a valid palindrome.
        return True
```

---

4. Complexity Analysis

Let `n` be the length of the string `s`.

Time Complexity: `O(n)`

Each character is visited at most once by either the `left` pointer or the `right` pointer.

Even though there are nested `while` loops, the total movement of both pointers is linear.

Space Complexity: `O(1)`

We do not create a new cleaned string.

We only use two integer pointers and a few temporary values.

---

5. Alternative Approaches

Alternative 1: Clean the String First

We can remove all non-alphanumeric characters, convert the remaining characters to lowercase, and then check whether the cleaned string equals its reverse.

```python
class Solution:
    def isPalindrome(self, s: str) -> bool:
        cleaned = ""

        for char in s:
            if char.isalnum():
                cleaned += char.lower()

        return cleaned == cleaned[::-1]
```

However, repeated string concatenation can be inefficient in Python because strings are immutable.

A better version is:

```python
class Solution:
    def isPalindrome(self, s: str) -> bool:
        cleaned = [char.lower() for char in s if char.isalnum()]
        return cleaned == cleaned[::-1]
```

Complexity

- Time Complexity: `O(n)`
- Space Complexity: `O(n)`

This approach is simpler to write, but it uses extra memory to store the cleaned string/list.

Key Pattern

This problem is a classic example of the **two pointers technique**.

Use two pointers when:

- You need to compare characters/items from both ends.
- You want to avoid extra memory.
- The problem involves symmetry, reversal, or palindrome checking.